

SMART CAMPUS & PLACEMENT PLATFORM

Project V2.1 — Database Migration & Schema Management

1. Flyway Database Migration

Flyway has been added to the Smart Campus & Placement Platform to provide **version-controlled database schema management**. It introduces a controlled way to create and evolve the PostgreSQL database as the application grows.

2. Why Flyway was added

During the initial development of the application, Hibernate used `spring.jpa.hibernate.ddl-auto=update` to automatically create or update the database schema from JPA entities. This is convenient during early development, but relying on automatic schema changes becomes harder to control and review as the number of entities and environments increases.

Flyway was therefore introduced so that database changes are represented explicitly as SQL migrations and can be maintained together with the application source code.

3. How Flyway works

Flyway uses versioned migration files. Each migration represents a specific database change and is executed in version order.

src/main/resources/db/migration/

V1__initial_schema.sql

V2__add_skill_table.sql

V3__add_company_table.sql

Flyway records applied migrations in a database table named **flyway_schema_history**. This allows the application and database to maintain a known migration history and schema version.

4. Flyway in this application

Configuration / component	Purpose
spring-boot-starter-flyway	Integrates Flyway with the Spring Boot application.
flyway-database-postgresql	Provides Flyway support for PostgreSQL.
V1__initial_schema.sql	Defines the initial Student and Project database schema.
flyway_schema_history	Stores Flyway migration history and schema version.
ddl-auto=validate	Allows Hibernate to validate the database schema without modifying it.

5. Existing Database Integration

Flyway was introduced after the PostgreSQL database already contained the Student and Project tables and application data. Instead of recreating the database, the existing schema was **baselined at version 1**.

The baseline established the starting migration version and allowed Flyway to begin tracking the existing database without deleting or recreating the existing tables.

The resulting migration history contains version 1 with a successful baseline, while the existing database remains intact.

6. Hibernate and Flyway Responsibilities

Flyway → Database schema evolution

Creates and changes tables, columns, constraints and other schema structures through versioned migrations.

Hibernate/JPA → Object-relational mapping and validation

Maps Java entities to database structures and verifies that the existing schema is compatible with the entity mappings.

This creates a clear separation between the application's object model and the controlled evolution of the database schema.

7. Configuration

The application now uses:

spring.jpa.hibernate.ddl-auto=validate

spring.flyway.baseline-on-migrate=true

spring.flyway.baseline-version=1

The baseline settings were used to adopt the existing database. Future schema changes will be introduced through new Flyway migrations rather than automatic Hibernate schema updates.

8. Benefits for the Application

- Database changes become explicit and version controlled.
- Schema changes can be reviewed alongside application code in Git.
- The same migration sequence can be applied consistently across environments.
- The database has a persistent record of its migration history.
- Hibernate no longer silently modifies the schema.
- Future domain expansion can introduce new tables and schema changes through controlled migrations.

9. Migration Principle

An applied migration becomes part of the database history. Future changes should normally be represented by new migration versions rather than editing an already-applied migration.

For example, adding the Skill domain will introduce a new migration such as **V2__add_skill_table.sql**, allowing the database schema to evolve in a controlled sequence.

Technology added: Flyway

Database: PostgreSQL

ORM: Hibernate / JPA

Schema management: Flyway migrations